

Summary

Instruction formats and execution dynamics are central to understanding processor efficiency in Computer Organization and Architecture (COA). A processor's performance is directly influenced by its Instruction Set Architecture (ISA), which defines how instructions are fetched, decoded, and executed.

Key insights include:

- The Program Counter (PC) always holds the address of the *next* instruction and increments immediately after instruction fetch.
- Instructions are functionally classified into Data Transfer, Data Processing, and Branch Control.
- Instruction formats range from 0-address (stack-based) to 3-address architectures, each offering different trade-offs.
- Memory addressability (byte vs. word) directly affects address field size and memory usage.
- Logical operations (AND, OR, XOR) enable precise bit-level manipulation without disturbing other data bits.

I. Instruction Execution Cycle

Every instruction executed by the CPU passes through three fundamental stages:

1. Fetch
2. Decode
3. Execute

Key Processor Registers Involved

- **Program Counter (PC)**
 - Holds the address of the instruction to be fetched
 - Increments immediately after fetch
 - Ensures sequential execution by default
- **Instruction Register (IR)**
 - Stores the fetched instruction
- **Instruction Decoder**
 - Interprets the instruction stored in IR
 - Determines operation type and operand count

Sequential Execution vs. Branch Execution

- **Sequential Execution:**
 - Occurs naturally due to automatic PC increment
- **Branch Execution:**
 - Branch instructions manually modify the PC
 - Enables jumps, loops, and conditional execution

II. Functional Classification of Instructions

Instructions are categorized based on their operational purpose:

Category	Function	Examples
Data Transfer	Moves data without modification	MOVE R1, R2; LOAD R0, 20
Data Processing	Performs computation	ADD R0, R1, R2; XOR R1, R2
Branch Control	Changes program flow	JUMP 2000; CALL; RET

Bit Manipulation Using Logical Instructions

Logical instructions are commonly used to manipulate individual bits:

- **AND (Bit Clearing)**
 - Bit AND 0 → 0
 - Bit AND 1 → unchanged
- **OR (Bit Setting)**
 - Bit OR 1 → 1
 - Bit OR 0 → unchanged
- **XOR (Bit Complementing)**
 - Bit XOR 1 → inverted
 - Bit XOR 0 → unchanged

III. Instruction Address Formats

The number of operands specified in an instruction defines its address format.

0-Address Instructions (Stack-Based)

- Operate implicitly on stack elements
- No operand fields specified
- Example: 8087 Math Co-processor (FADD uses ST0 and ST1)

- Advantage: Smaller instruction size, useful for 80-bit floating-point operations

1-Address Instructions (Accumulator-Based)

- One explicit operand
- One implicit operand (Accumulator)
- Example: 8085
 - $\text{ADD B} \rightarrow \text{Accumulator} = \text{Accumulator} + \text{B}$
- Results accumulate in a single register

2-Address Instructions

- Two operands specified
- First operand acts as both source and destination
- Example: 8086
 - $\text{ADD BL, CL} \rightarrow \text{BL} = \text{BL} + \text{CL}$

3-Address Instructions

- Two source operands and one destination operand
- Example: ARM
 - $\text{ADD R0, R1, R2} \rightarrow \text{R0} = \text{R1} + \text{R2}$
- Advantage: Source registers remain unchanged, offering maximum flexibility

IV. Instruction Word Structure and Bit Calculations

An instruction consists of:

- Opcode
- Operand fields (registers, memory addresses, or immediate data)

Opcode Size Calculation

If the processor supports 2^n instructions:

Opcode size = n bits

Example:

- 50 instructions $\rightarrow$ 6-bit opcode
(since $2^6 = 64$)

Register Field Size

To address 2^n registers:

Register field size = n bits

Example:

- 64 registers $\rightarrow$ 6 bits per register field

V. Memory Addressability

Instruction address field size depends on how memory is organized.

Byte-Addressable Memory

- Each address refers to 1 byte
- Higher number of addresses required

Word-Addressable Memory

- Each address refers to 1 word (2, 4, or 8 bytes)
- Fewer addresses required

Example:

- Total memory = 64 bytes
- Word size = 2 bytes
- Number of words = 32
- Address bits required = 5

(Byte-addressable system would require 6 bits)

VI. Immediate Data Representation

The size of the immediate field determines the value range:

- **Unsigned n -bit field:**
Range = 0 to $(2^n - 1)$
- **Signed n -bit field:**
Range split between positive and negative values
Example:
 - 8-bit signed $\rightarrow$ -128 to +127

VII. Memory Alignment and Storage Efficiency

Byte-Aligned Storage

- Instructions must occupy whole bytes
- Fractional bytes are not allowed

Example:

- Logical instruction size = 34 bits
- Storage required = 5 bytes (40 bits)
- Results in internal fragmentation

Program Size Calculation

Total Memory Consumed =

(Number of Instructions) × (Aligned Instruction Size in Bytes)